

Prerequisites

From the first week of this course you **build and run C/C++ programs on your own computer**, run your project on **Git and GitHub**, and prove your security controls with **unit tests**. You therefore need to arrive with the knowledge and tools below. The formal prerequisite of the course is **CEN107 Algorithms and Programming I**; in particular, the development environment, Git, unit testing and template usage taught in the first weeks of that course are **assumed** in this course.

⚠ Before the first class

Do the checks in [Section 5](#) on your own computer. If you get stuck on a step, go back to the corresponding Algorithms and Programming I page; if you still cannot solve it, ask in the first class.

1. What we bring from Algorithms and Programming I (required)

Topic	Where is it taught?	Where is it used in this course?
Development environment: compiler (GCC/Clang/MSVC), IDE, WSL on Windows, building with CMake	CEN107 (CE103) Week 2 – Development environments	Every week's demos; lab setup in Week 1
Git and GitHub: creating a repository, <code>clone</code> , <code>commit</code> , branches, <code>pull request</code> , <code>.gitignore</code>	CEN107 (CE103) Week 3 – Version management with Git	Project guide (plan, setup, submission); change management in Week 1; version identity in Week 12
Unit testing and coverage tools: writing tests, running them with <code>ctest</code> , coverage reports	CEN107 (CE103) Week 4 – Unit testing and libraries	S16 test results in the project; sanitizers and fuzzing in Week 4 ; test plan in Week 12
Using project templates: creating a private repository from a template ("Use this template", not fork), building it, producing its tests and documentation	CEN107 (CE103) Weeks 2–4	Project guide · 2. Project setup – cpp-cmake-ctest-template

Optional starting point: [CEN107 \(CE103\) Week 1 – Introduction and developer roadmap](#).

2. C programming basics (required)

Almost all demos are written in C, and security bugs mostly come from misusing these basic constructs. You should be able to read and write the following comfortably:

- pointers, arrays and strings (`char[]`, the `\0` terminator),
- dynamic memory (`malloc`, `calloc`, `free`) and structures (`struct`),
- reading and writing files (`fopen`, `fread`, `fgets`),

- compiling a program with several source files and using command-line arguments (`argc` , `argv`).

These are used heavily throughout [Week 1](#) (memory, overflows) and [Week 4](#) (C/C++ hardening).

3. Background briefly recalled in class

It is fine if you do not know these yet; the "**Before we start**" section of the relevant week gives a short recap. Looking at them in advance makes the class easier.

Background	Needed in which week?
Terminal: <code>cd</code> , <code>ls</code> , file paths and environment variables in PowerShell and Linux/WSL	Week 1 and all demos
Binary and hexadecimal numbers, bytes, XOR	Weeks 3 , 9 , 11
Basic networking: client–server, TCP/IP, HTTP	Weeks 3 and 10
Simple SQL (<code>SELECT ... WHERE</code>), basic Java or Python syntax	Week 5

4. What must be installed on your computer

A laptop is required. Come to the first class with these tools installed:

- A C/C++ compiler: **Visual Studio 2022** (Desktop development with C++) on Windows, **GCC** or **Clang** on Linux/WSL
- **CMake**, **Git** and a **GitHub** account
- **WSL2 + Ubuntu** if you use Windows
- **OpenSSL 3**, **SQLite**, **Python 3**, **JDK 21**

GoogleTest, SoftHSM2 and Tigress are also used during the term; their installation is explained in the relevant week.

Setup guide for the demo code: [code/README.en.md](#) .

5. Check yourself (before the first class)

Each of the commands below should print a version number (the numbers may differ):

```
git --version           # git version 2.x
cmake --version         # cmake version 3.2x or later
wsl --status            # Default Distribution: Ubuntu

git --version           # git version 2.x
cmake --version         # cmake version 3.2x or later
gcc --version           # gcc (Ubuntu ...) 11 or later
openssl version         # OpenSSL 3.x
```

Then create your own private repository from the project template ("Use this template → Create a new repository → Private", not fork), build it and run its tests:

```
git clone https://github.com/<your-username>/cpp-cmake-ctest-template.git
cd cpp-cmake-ctest-template
cmake -S . -B build
cmake --build build
ctest --test-dir build      # expected: 100% tests passed
```

If all of these steps finish without errors, you are ready for the course.